REPLUG: Retrieval-Augmented Black-Box Language Models

Enhancing Large Language Models without Parameter Access

Based on the paper by: Shi et al. (2023)

Presentation Prepared by : Ali Sezer Uysal

Presentation Outline

- **Introduction**
  - State of LLMs and Problem Definition
  - •Limitations & Black-Box Constraint
- **Methodology: REPLUG Framework**
  - Retrieve and Plug Concept
  - Two Stages: Inference & Training
  - LM-Supervised Retrieval (LSR)
- **Experiments & Results**
  - Language Modeling Performance
  - Downstream Tasks (MMLU & QA)
  - Evaluation Metrics
- **Analysis& Discussion**
  - Why Ensemble Works
  - Generality Across Model Families
  - Impact on Rare Entities
- **Conclusion &Future Directions**
  - Summary of Contributions
  - 1.Limitations & Next Steps

Introduction: The State of LLMs and Problem Definition

Large Language Models (LLMs)

- Models like **GPT-3** and **Codex** have demonstrated impressive performance across a wide range of language tasks.
- **Mechanism:** These models store a substantial amount of world and domain knowledge implicitly within their parameters.

The Limitations

- **Hallucination:** They are prone to generating false or fabricated information.
- **Static Knowledge:** They can’t represent the full "long tail" of knowledge and fail to update with new information after training.
- **Solution:** Retrieval-Augmented Generation (RAG) can address these issues by retrieving knowledge from external datastores.

The Problem: Black-Box & Scale

The Accessibility Challenge

- State-of-the-art LLMs (>100B parameters) are typically deployed as

  Black-Box APIs (e.g., GPT-3, Codex).

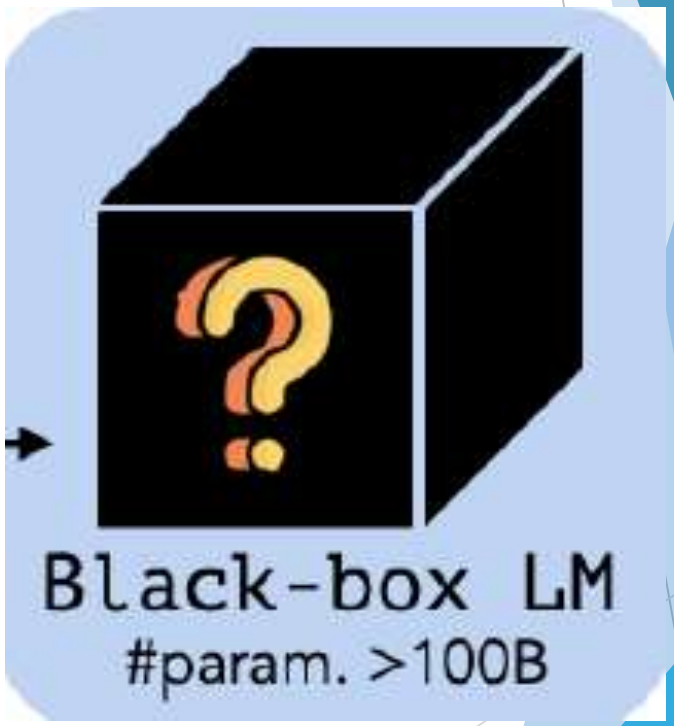
- **Constraints:**
  - No access to internal parameters or gradients.
  - No access to internal representations (embeddings).
  - Fine-tuning is often not supported or restricted.

The Computational Barrier

- Even open-source large models (e.g., BLOOM-176B) are difficult to fine-tune locally.
  - *Example:* Fine-tuning BLOOM requires approx. 72 A100 GPUs ($15k each).

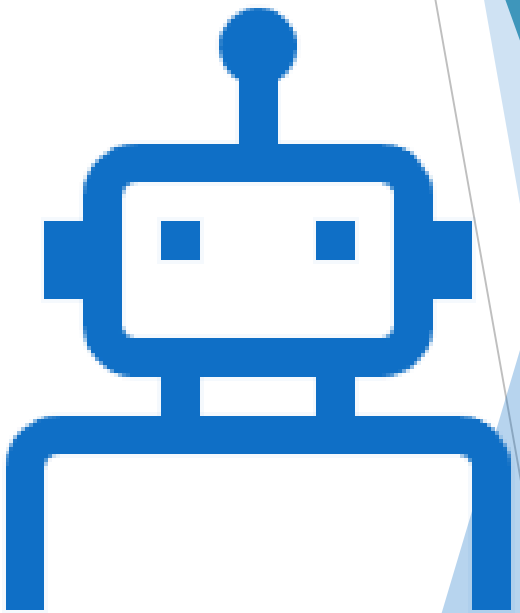
Failure of Existing RAG Methods

- Traditional methods (e.g., Atlas, RETRO) require **White-Box** access to update LM parameters, which is impossible for API-based models



The Proposed Solution: REPLUG

- **Core Concept: "Retrieve and Plug"**
- Treats the Language Model as a **frozen Black Box**.
- Augments the LM with a **tuneable retrieval model** that can be plugged in easily.
- **Key Contributions**
- **REPLUG (Inference):** A framework that retrieves relevant documents and prepends them to the input for the black-box LM .
- **REPLUG LSR (Training):** A novel training scheme ("LM-Supervised Retrieval") that tunes the retriever using the frozen LLM as a supervisor.
- **Impact:** The first work to show retrieval benefits for LLMs >100B parameters (GPT-3, Codex) without accessing their weights.

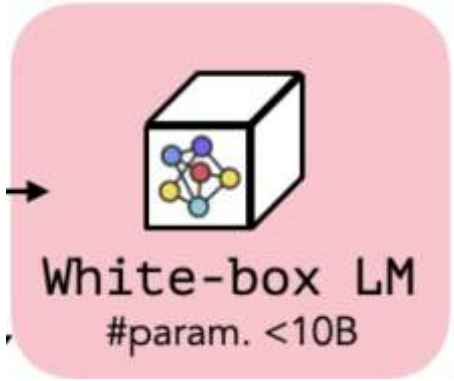


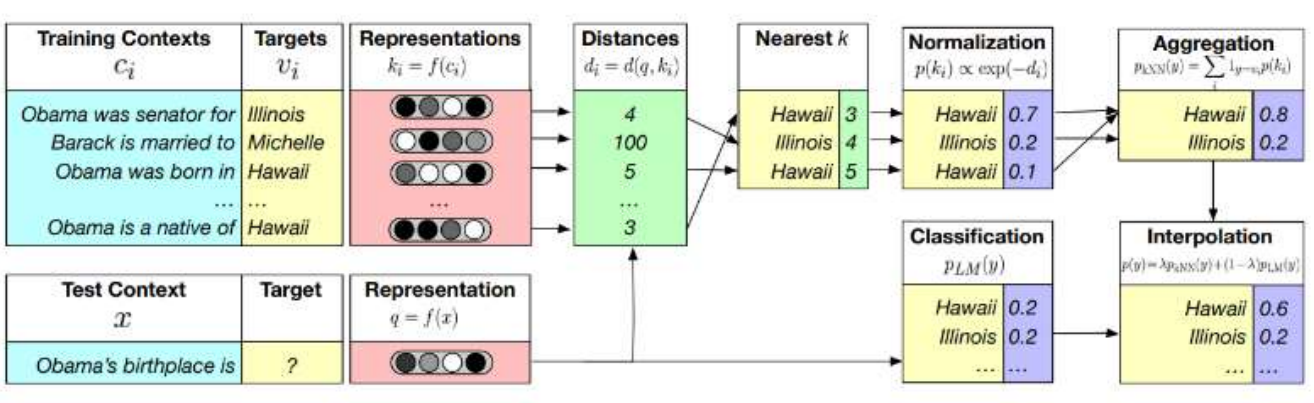
Why Existing Methods Fail on Black-Box Models

- **White-Box RAG (e.g., Atlas, RETRO):**
  - ***Mechanism**:* Modifies the internal architecture (Cross-Attention) or fine-tunes the LM using gradients.
  - ***Limitation**:* Cannot be applied to API-based models where gradients are inaccessible .
- **kNN-LM:**
  - ***Mechanism**:* Interpolates predictions using nearest neighbors from a datastore.
  - ***Limitation**:* Requires access to internal **LM embeddings** (hidden states), which commercial APIs rarely provide .

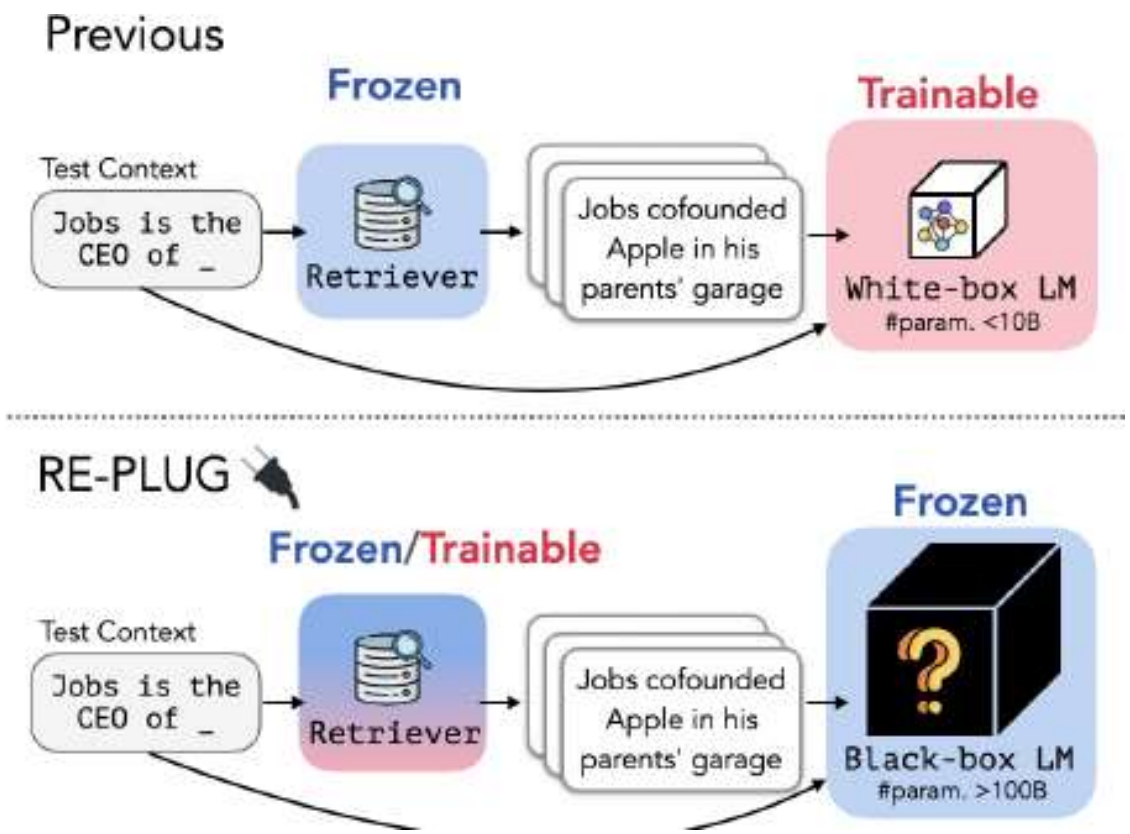
REPLUG's Unique Position

- It is the first framework designed specifically for **Black-Box** settings.
- It works by only manipulating the **input** (prompt) and the **retriever**, without touching the LM's weights.





REPLUG: The Two Main Stages



Overview of the Framework

REPLUG consists of two distinct stages that handle the "Retrieve and Plug" paradigm:

1. Inference Stage (The "Plug" Part)

- How to use retrieved documents with a frozen Black-Box LM to generate text.
- Uses a novel **Ensemble Scheme** to bypass context window limits.

2. Training Stage (The "Tuning" Part)

- How to improve the retriever using the Black-Box LM as a supervisor.
- Introduced as **REPLUG LSR** (LM-Supervised Retrieval)

Stage 1: Inference - Document Retrieval

The Retrieval Process

•**Goal:** Find a small set of documents (d) relevant to the input context (x) from a massive external corpus.

•**Architecture:** Uses a **Dual Encoder** (Contriever).

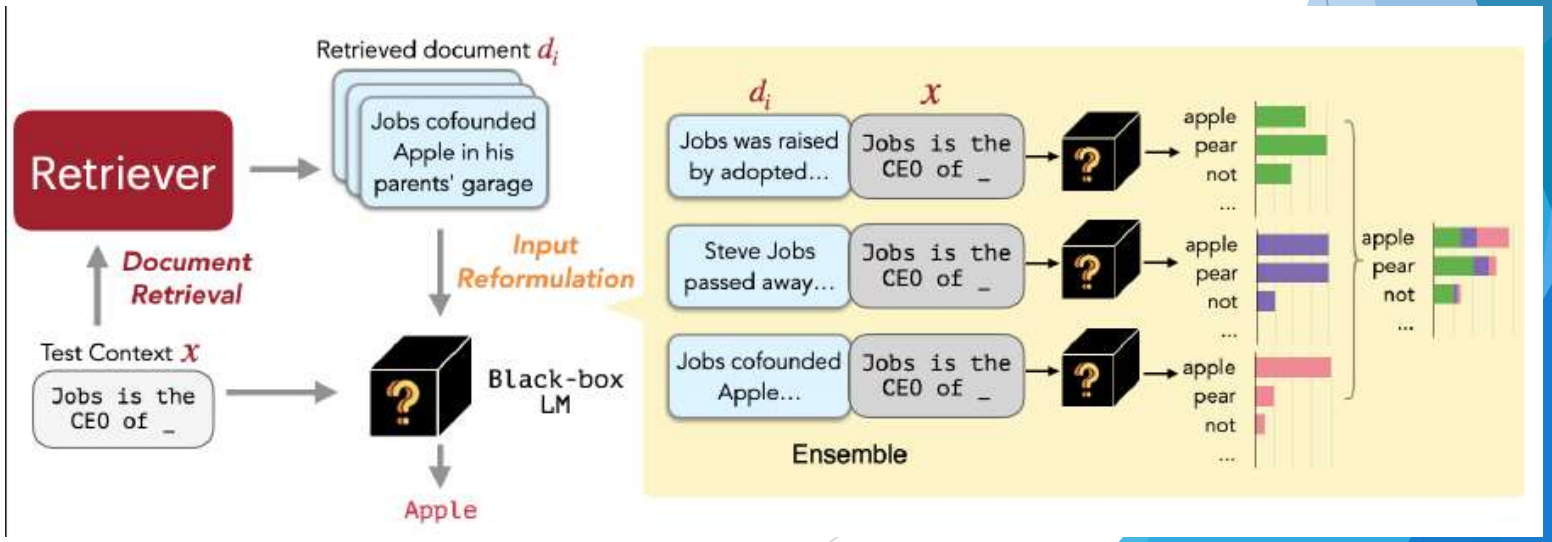
- •Encodes both the input context and documents into dense vectors [5].

•**Similarity Search:**

- •Computes similarity using **Cosine Similarity**:

$$s(d, x) = \cos(E(d), E(x))$$

- •Uses **FAISS** index for efficient, large-scale search[6].



Stage 1: Inference - The Ensemble Scheme

The Context Window Challenge

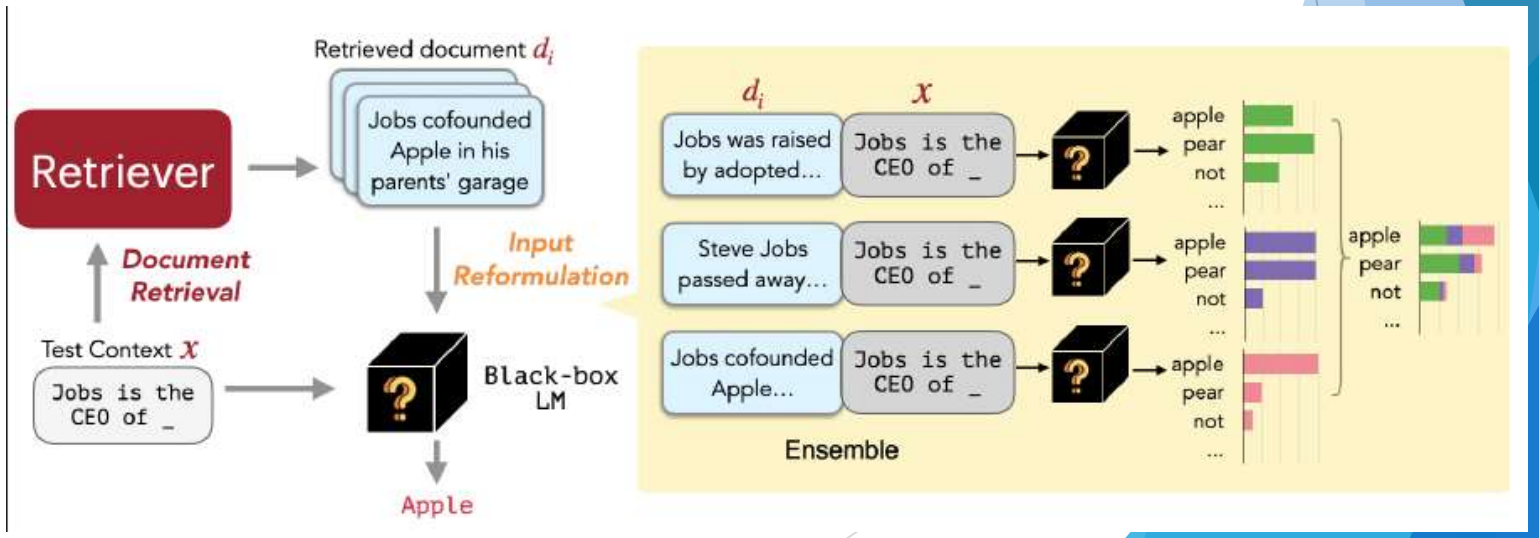
•Simply prepending all retrieved documents to the input often exceeds the LM's maximum context length (e.g., 2048 or 4096 tokens)[7].

Solution: Parallel Input Reformulation

•Instead of concatenation, REPLUG processes documents in **parallel**:

- •**Pass 1:** [Document 1] + [Input Context] -> LM -> Probability 1
- •**Pass 2:** [Document 2] + [Input Context] -> LM -> Probability 2
- •...
- •**Pass k:** [Document k] + [Input Context] -> LM -> Probability k

•This allows using an arbitrary number of documents without hitting the context limit.



Stage 1: Inference - Mathematical Aggregation

Weighted Ensemble Prediction

•The final probability of the next token (y) is a weighted average of the probabilities from all passes.

Formula:

$$p(y|x, D') = \sum_{d \in D'} p(y|d \circ x) \cdot \lambda(d, x)$$

•**Explanation:**

- $p(y|d \circ x)$: The LM's prediction based on a specific document.
- $\lambda(d, x)$:The weight of that document, based on its retrieval similarity score (Softmax of retrieval scores) [8].

•**Result:** The model "votes" using information from multiple documents simultaneously.

Stage 2: Training - REPLUG LSR

The Need for Training

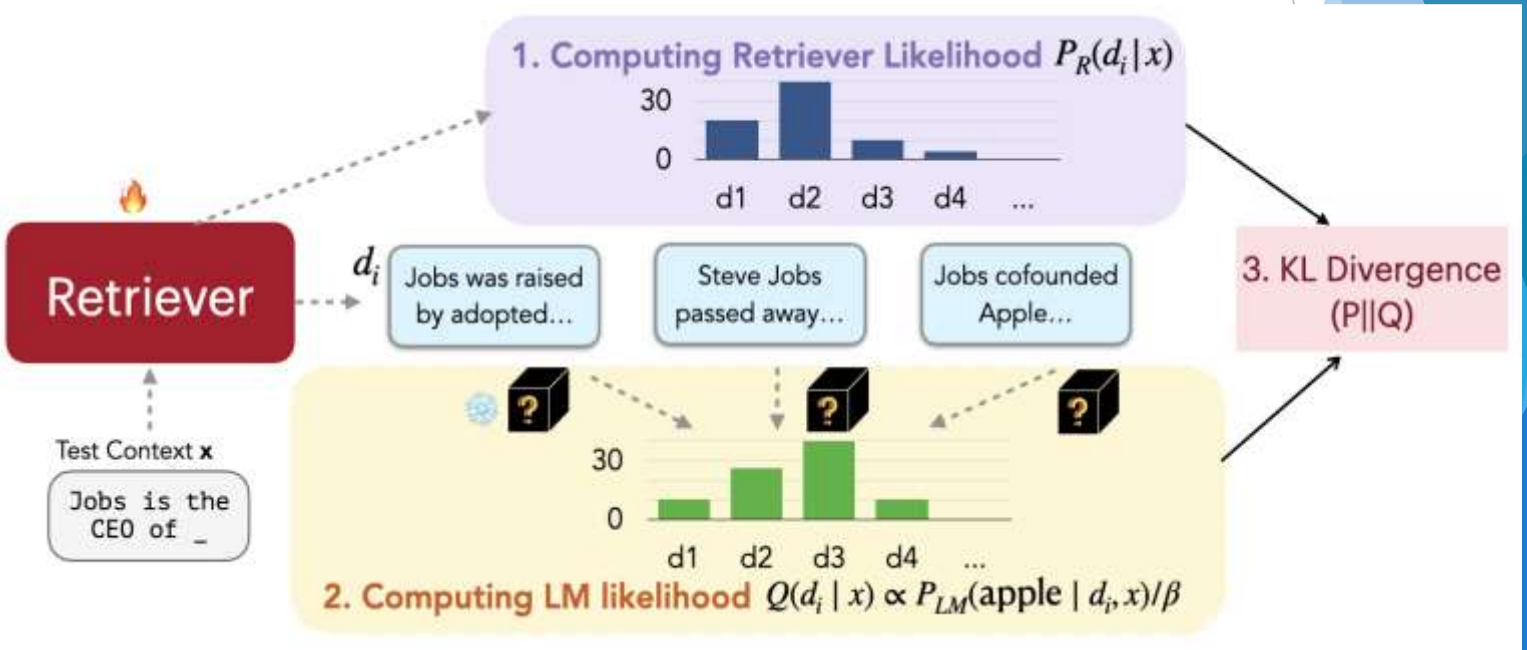
While the inference scheme works with any retriever, performance can be improved by tuning the retriever specifically for the target LLM.

Challenge: Since the LLM is a Black Box, we cannot backpropagate gradients through it to train the retriever.

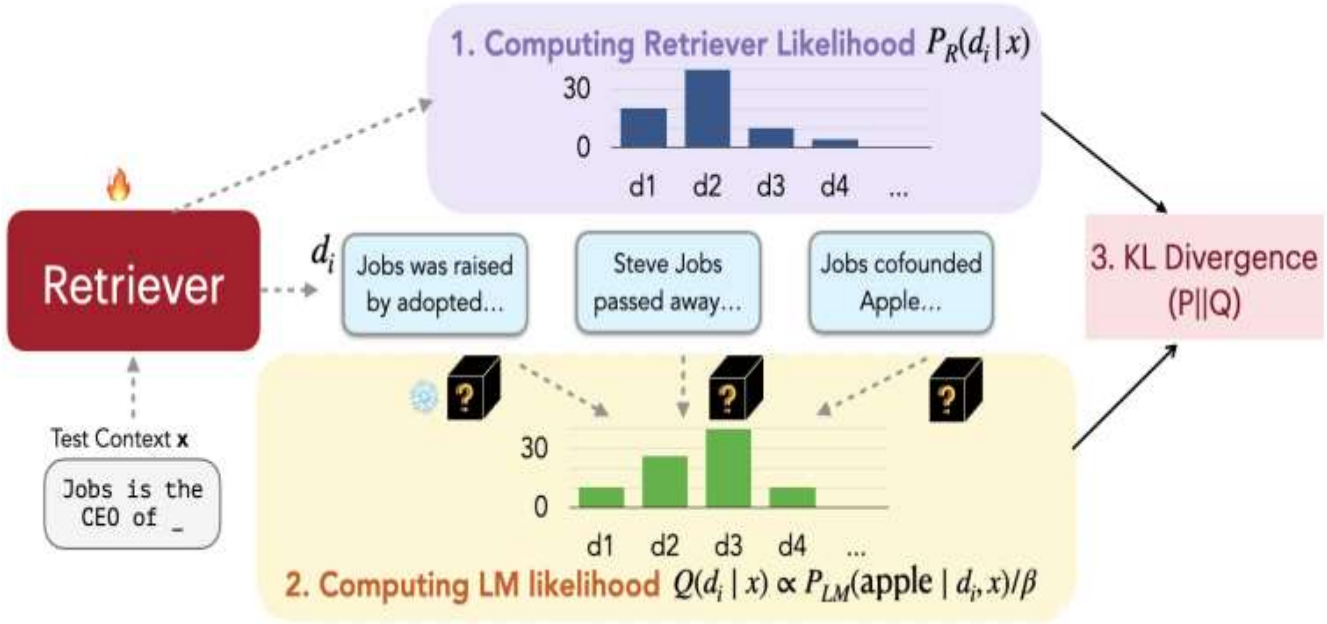
Solution: REPLUG LSR (LM-Supervised Retrieval)

Concept: Use the frozen Black-Box LLM as a "Supervisor" or "Scoring Function".

Goal: Train the retriever to assign high probabilities to documents that actually help the LLM reduce perplexity (better predict the ground truth).



Training Step 1 & 2: Computing Scores



Step 1: Retriever Likelihood

•The retriever retrieves top-k documents and assigns a probability distribution based on similarity scores.

•Formula:

$$P_R(d|x) = \frac{e^{s(d,x)/\gamma}}{\sum_{d' \in D'} e^{s(d',x)/\gamma}}$$

•This represents what the **Retriever thinks** is relevant.

Step 2: LM Likelihood

We feed each document to the frozen LLM and measure the probability of the ground truth output y.

•Formula:

$$Q(d|x,y) = \frac{e^{P_{LM}(y|d,x)/\beta}}{\sum_{d' \in D'} e^{P_{LM}(y|d',x)/\beta}}$$

•This represents what the **LLM confirms** is helpful.

Training Step 3: Optimization

Minimizing the Gap (KL Divergence)

•We want the Retriever's opinion to match the LLM's reality.

• Loss Function:

$$\mathcal{L} = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} KL(P_R(d|x) || Q_{LM}(d|x, y))$$

The "Black-Box" Constraint

•During this optimization, the **LLM parameters are fixed**.

•We only update the **Retriever's parameters**.

•This effectively "distills" the LLM's preference into the Retriever

Training Step 4: Index Maintenance

The Stale Index Problem

- As the retriever is trained, its encoder parameters change.
- Consequently, the pre-computed embeddings of the 36M documents in the index become outdated ("stale").

Asynchronous Update Strategy

- Re-computing embeddings for the entire corpus at every step is computationally impossible.

- **Solution:**
  - Update the retriever for T steps (e.g., 3,000 steps).
  - Pause training to **re-encode the corpus** and **rebuild the index**.
  - Resume training with the fresh index [3].

ar

Experimental Setup - Model Configuration

1. The Retriever Backbone

Choice: Contriever (Unsupervised Dense Retriever).

Reasoning: It provides strong initial performance compared to sparse methods (BM25) and other dense retrievers.

Initialization: Pre-trained weights are used as the starting point for REPLUG LSR.

2. The Supervisor LLM (For Training)

Choice: GPT-3 Curie (~6.7B parameters).

Reasoning:

While larger models (Davinci) are better, they are too expensive/slow to query millions of times during the training loop.

Curie offers the best balance between signal quality and computational efficiency.

3. The Inference LLMs (For Testing)

GPT-3 Family: Ada, Babbage, Curie, and **Davinci (175B)**.

Codex: Used specifically for MMLU and QA tasks due to its superior reasoning capabilities.

Experimental Setup - Training Details Data & Corpus

- **Training Query Data:** 800k sequences sampled from **The Pile**.
- **Retrieval Corpus:** 36M documents (128 tokens each) sampled from The Pile.
- **Decontamination:** Crucially, the authors ensure no overlap between the query data and the retrieval corpus to prevent "trivial retrieval" (cheating by finding the exact text).

Hyperparameters

- **Training Duration:** 25,000 steps.
- **Index Refresh Rate:** Every **3,000 steps** (to handle stale embeddings).
- **Batch Size:** 64.
- **Learning Rate:** 2e-5 (Adam Optimizer).
- **Retrieval Count:** Top-20 documents retrieved for each query during training.
- **Temperatures:**
  - Retriever Softmax Temperature :0.1
  - LM Softmax Temperature :0.1

Experimental Setup - Evaluation Metrics

Evaluation Tasks

Language Modeling:

- *Dataset:* The Pile (Test split).
- *Metric:* **BPB (Bits Per Byte)**.
- *Why BPB?* To allow fair comparison between models using different tokenizers (e.g., GPT-2 vs. GPT-3).

MMLU (Massive Multi-task Language Understanding):

- *Metric:* Accuracy (%).
- *Setting:* **5-shot** in-context learning.

Open-Domain QA:

- *Datasets:* Natural Questions (NQ), TriviaQA.
- *Metric:* **Exact Match (EM)**.
- *Setting:* Few-shot (e.g., 64-shot or 16-shot).

Language Modeling Results

Model		# Parameters	Original	+ REPLUG	Gain %	+ REPLUG LSR	Gain %
GPT-2	Small	117M	1.33	1.26	5.3	1.21	9.0
	Medium	345M	1.20	1.14	5.0	1.11	7.5
	Large	774M	1.19	1.15	3.4	1.09	8.4
	XL	1.5B	1.16	1.09	6.0	1.07	7.8
GPT-3 (black-box)	Ada	350M	1.05	0.98	6.7	0.96	8.6
	Babbage	1.3B	0.95	0.90	5.3	0.88	7.4
	Curie	6.7B	0.88	0.85	3.4	0.82	6.8
	Davinci	175B	0.80	0.77	3.8	0.75	6.3

- **Metric:** BPB (Bits Per Byte) - *Lower is better.*
- **Scope:** Evaluated on 8 different models from GPT-2 and GPT-3 families.
- **Key Finding 1:** Consistent improvement across **all** model sizes.
- **Key Finding 2:** Even the largest model (**Davinci 175B**) gains **6.3%** improvement.
- **LSR Effect:** Training the retriever (LSR) adds significant value over standard REPLUG.

Massive Multi-task Language Understanding (57 subjects: Math, Law, History...).

Model	# Parameters	Humanities	Social.	STEM	Other	All
Codex	175B	74.2	76.9	57.8	70.1	68.3
PaLM	540B	77.0	81.0	55.6	69.6	69.3
Flan-PaLM	540B	-	-	-	-	72.2
Atlas	11B	46.1	54.6	38.8	52.8	47.9
Codex + REPLUG	175B	76.0	79.7	58.8	72.1	71.4
Codex + REPLUG LSR	175B	76.5	79.9	58.9	73.2	71.8

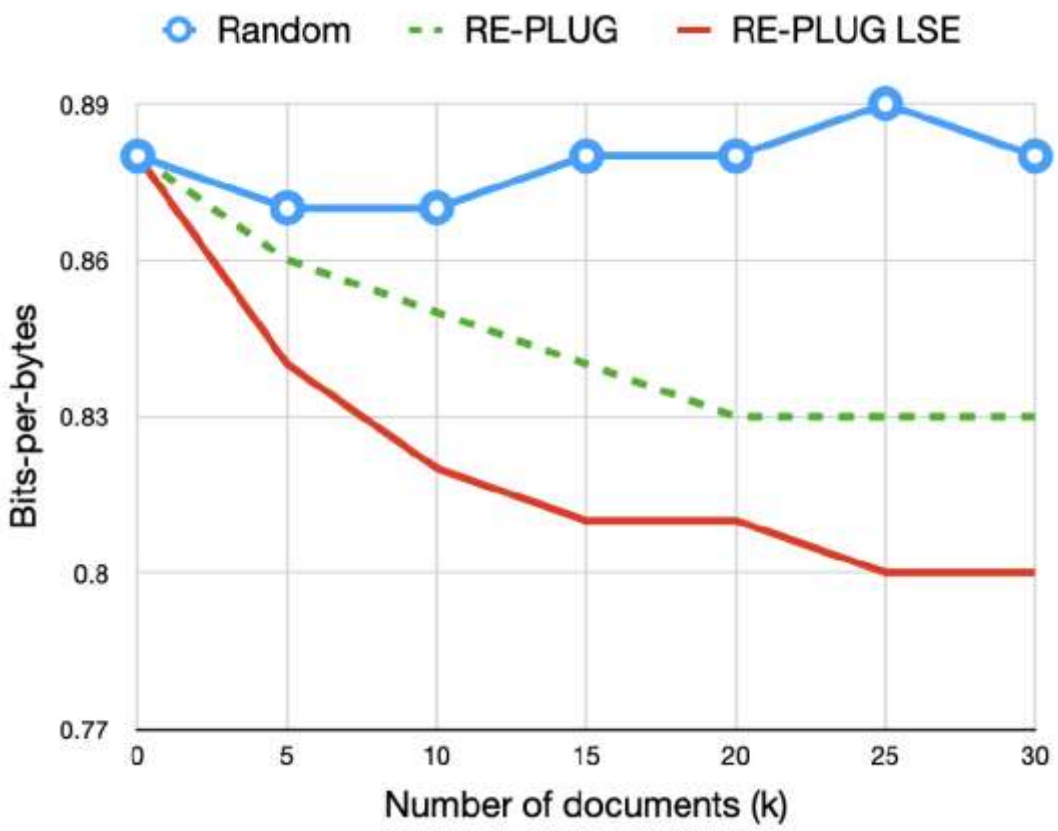
- **Model:** Codex (175B).
- **Comparison:**
  - **vs. Baseline:** +3.5% absolute gain over original Codex.
  - **vs. SOTA RAG:** Significant lead over **Atlas (11B)**.
  - **vs. Giants:** Comparable performance to **Flan-PaLM (540B)** despite being 3x smaller.
- **Insight:** Retrieval improves reasoning (STEM), not just knowledge memorization.

Open-Domain QA Results

Model	NQ		TQA	
	Few-shot	Full	Few-shot	Full
Chinchilla	35.5	-	64.6	-
PaLM	39.6	-	-	-
Codex	40.6	-	73.6	-
RETRO†	-	45.5	-	-
R2-D2†	-	55.9	-	69.9
Atlas†	42.4	**60.4**	74.5	**79.8**
Codex + Contriever$_{cc}$[2]	44.2	-	76.0	-
Codex + REPLUG	44.7	-	76.8	-
Codex + REPLUG LSR	**45.5**	-	**77.3**	-

•**Datasets:** Natural Questions (NQ) & TriviaQA.
•**Setting:** Few-shot (16 examples).
•**Result:**
•**NQ: +12.0%** relative improvement over Codex.
•**TriviaQA: +5.0%** relative improvement.
•**Significance:** Achieves new **State-of-the-Art (SOTA)** for few-shot QA, outperforming fine-tuned white-box models like Atlas.

Analysis - Why Ensemble Works?



Graph Overview

X-Axis: Number of retrieved documents (k).

Y-Axis: Bits-per-byte (BPB) on GPT-2 Medium.

Note: **Lower is better** (Lower BPB = Better Prediction).

Key Observations

Blue Line (Random Documents):

Retrieving random documents and ensembling them **increases** BPB (Performance degrades).

Implication: The ensemble mechanism itself does not improve performance; feeding "noise" confuses the model.

Green Dashed Line (REPLUG - Standard):

Using a standard dense retriever (Contriever) improves performance significantly as k increases.

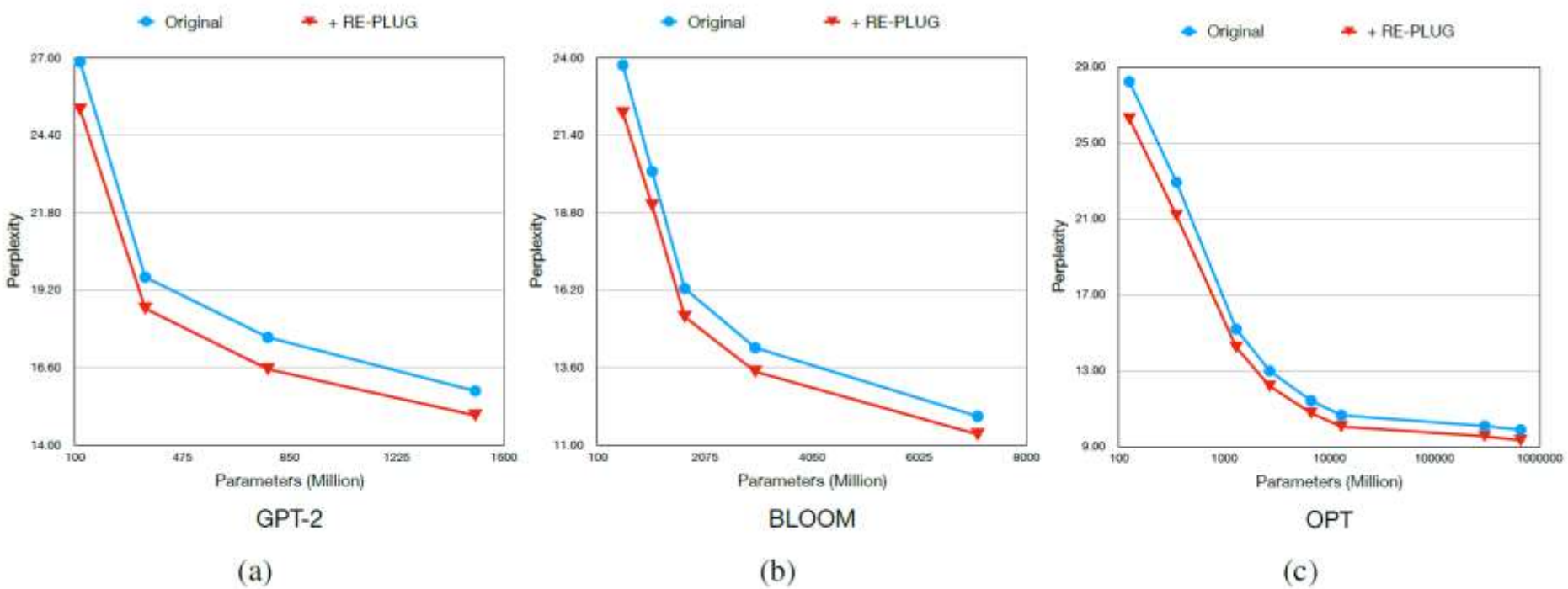
Implication: Relevance matters.

Red Line (REPLUG LSR - Ours):

The tuned retriever achieves the **lowest BPB**.

Saturation Point: Performance gains plateau around **k=10**, suggesting that retrieving 10 documents is the efficiency "sweet spot".

Analysis - Generality Across Model Families



Overview

Goal: To verify if REPLUG is model-agnostic (works on any architecture).

Key Observations (Figure 5)

Consistent Gain: In all three charts, the **REPLUG (Red Line)** is consistently below the **Baseline (Blue Line)**.

Scalability: The improvement persists from the smallest models (~100M) up to massive models (66B).

Parallel Trend: The scaling curves remain parallel, meaning retrieval does not lose its effectiveness as the model gets larger.

Qualitative Analysis - Rare Entities (Case Study)

Visual Breakdown (Figure 6)

Retrieved Document (Top Box):

- REPLUG finds an external document explicitly stating: *"Two of China's most famous poets, **Li Bai** and Du Fu..."*

Continuation (Bottom Box):

- Using this document, the model predicts: *"Along with **Li Bai**..."*

Perplexity Drop (The Result):

- **Li Bai:** Perplexity improves (drops) by **15%**.
- **Greatest:** Perplexity improves by **5%**.

Conclusion

- Retrieval acts as a dynamic memory for long-tail knowledge, significantly boosting confidence on specific names and facts that the model's static parameters might miss.

Conclusion & Future Directions

1. Summary of Contributions

- **REPLUG Paradigm:** We introduced a novel retrieval-augmented framework that treats the Language Model as a **Black Box**, augmenting it with a tuneable retrieval model.
- **Universality:** Our evaluation demonstrates that REPLUG can be seamlessly integrated with **any existing language model** to enhance performance on both language modeling and downstream tasks.
- **Impact:** This work proves that even state-of-the-art, large-scale LMs (like GPT-3) are not self-sufficient and benefit significantly from external retrieval.

2. Limitations & Future Work

- **The Interpretability Challenge:** A key limitation is the lack of transparency; it remains unclear when the model relies on **retrieved knowledge** versus its own **parametric memory**.
- **Future Research:** Next steps should focus on developing retrieval-augmented models that are not only accurate but also **interpretable**, allowing us to understand the source of their decisions.